

# Embedded Systems Laboratory

## Assignment Manual

Jan Broenink, Peter Kingma, Jos Vos, Vincent Groenhuis and Bas De Quinze

Monday 18<sup>th</sup> May, 2026



# Contents

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                | <b>1</b>  |
| 1.1      | Context . . . . .                                  | 1         |
| 1.2      | Equipment . . . . .                                | 1         |
| 1.3      | Organisation . . . . .                             | 2         |
| 1.4      | Responsibility and fraud; Using AI tools . . . . . | 3         |
| 1.5      | Document History and Disclaimer . . . . .          | 4         |
| <b>2</b> | <b>Getting familiar with the tools</b>             | <b>5</b>  |
| 2.1      | Raspberry Pi and icoBoard Tools . . . . .          | 5         |
| 2.2      | Altera tools . . . . .                             | 9         |
| <b>3</b> | <b>Introduction to the logic analyser</b>          | <b>12</b> |
| 3.1      | Goals . . . . .                                    | 12        |
| 3.2      | Tools . . . . .                                    | 12        |
| 3.3      | Assignment . . . . .                               | 12        |
| <b>4</b> | <b>Building-blocks tutorial</b>                    | <b>15</b> |
| 4.1      | Raspberry Pi and icoBoard . . . . .                | 15        |
| 4.2      | ARM – the one on the Raspberry Pi . . . . .        | 15        |
| 4.3      | Cyclone 5 on DE10-Nano . . . . .                   | 18        |
| <b>5</b> | <b>Debugging SPI with the Logic Analyzer</b>       | <b>22</b> |
| 5.1      | Goals . . . . .                                    | 22        |
| 5.2      | Tools . . . . .                                    | 22        |
| 5.3      | Assignment . . . . .                               | 22        |
| <b>6</b> | <b>Combining the hardware</b>                      | <b>23</b> |
| 6.1      | Goals . . . . .                                    | 23        |
| 6.2      | Assignments . . . . .                              | 23        |
| 6.3      | Links . . . . .                                    | 24        |
| <b>7</b> | <b>Influence physical system behavior</b>          | <b>25</b> |
| 7.1      | Goals . . . . .                                    | 25        |
| 7.2      | Assignments . . . . .                              | 25        |
| <b>8</b> | <b>Design the final demo</b>                       | <b>26</b> |
| 8.1      | Goals . . . . .                                    | 26        |
| 8.2      | Assignments . . . . .                              | 26        |

|                                    |           |
|------------------------------------|-----------|
| <b>A Working with the setup</b>    | <b>27</b> |
| A.1 Connections . . . . .          | 28        |
| <b>B Verilog Simulation</b>        | <b>29</b> |
| B.1 Installing the tools . . . . . | 29        |
| B.2 Using the tools . . . . .      | 29        |
| <b>C H-bridge</b>                  | <b>31</b> |
| <b>Bibliography</b>                | <b>32</b> |

# 1 Introduction

## 1.1 Context

The goal of this practical work is to train yourself in practical embedded system design, and to perform Design Space Exploration (DSE) to experience consequences of computer-architectural choices. The focus is indeed on the embedded part of the cyber-physical system.

To reach this goal, the assignments start as real training assignments and end as the ultimate DSE challenge.

The provided computer hardware parts support various choices of architecture, structure of algorithms, etc. Two different embedded computer configurations are available, to have ample choices. However, for the training part (Chapters 2 – 4), we indicate which configuration can best be used.

## 1.2 Equipment

For the practical work, *JIWY* is available, a 2D pan-tilt platform, consisting of 3D printed parts. It has a camera attached, which is needed to control the setup via a vision-in-the-loop solution. Each group can use a *JIWY*, and work with that *JIWY* until the end of the practical work. This implies that you always use the *same* setup.

*JIWY* can be controlled using several computer-hardware configurations. We mention 3 possible configurations here:

- The Altera DE10-NANO, an FPGA development board, shown in Figure 1.1, of which *both* the FPGA (Cyclone 5) and the ARM processor (realised on chip in the so-called *HPS* – Hard Processor System) are used.
- The combination of an Altera DE10-NANO, Figure 1.1, and a Raspberry Pi, Figure 1.2. The Raspberry Pi is an ARM *Computer on Module* configured to use linux in this course.
- A Raspberry Pi with a Lattice FPGA as HAT (*Hardware on Top*) board connected via SPI bus (through the 40 pins GPIO connector) to the Raspberry Pi, as shown in Figure 1.3.

The main differences between the platforms are (1) the way of communication between the ARM processor and the FPGA, and (2) the brand and type of FPGA.

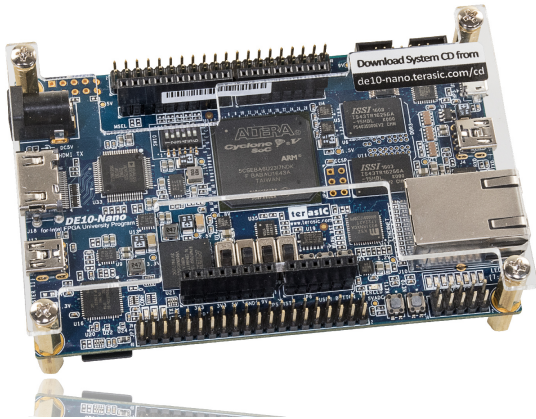
**Note:** you *must* use part of the given computer hardware. You are *not* allowed to use other computer hardware.

To develop programs for the FPGA and ARM processor you need a development toolset with a C compiler and the FPGA tool chains. For this, you can use VS Code with the indicated plugins. This FPGA tool chain runs on Linux. On your own laptop you can use a rather lightweight virtual machine (Multipass) to run the FPGA tool chain at home. Of course, you can run this tool chain on the Raspberry Pi in the Lab. However, you then have rather long compilation times (compared to an average modern laptop), and might waste Lab time.

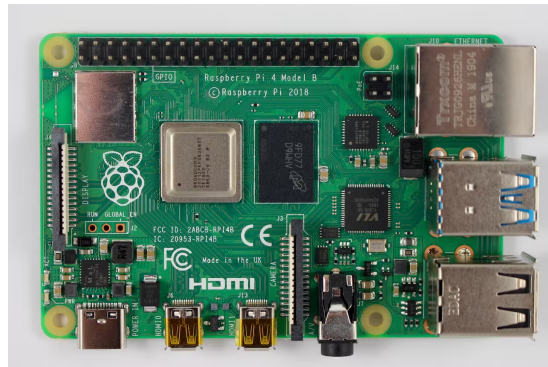
The first three chapters of this document need to be worked on with the indicated FPGA respectively ARM processor. Later, you can choose which processing node to use.

Next to this embedded hardware, each lab location has a unique setup with the embedded boards, the *JIWY* setup and a measurement PCB, to measure relevant signals on the PMOD connections between the embedded board and the *JIWY*.

Once your group has chosen a setup to work with, you have always to use that associated location.



**Figure 1.1:** Altera DE10-NANO development board <https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&No=1046>



**Figure 1.2:** Raspberry Pi computer on module <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>



**Figure 1.3:** Raspberry Pi with Lattice FPGA on icoBoardHAT. The PMOD connectors are labelled P1 to P4.

### 1.3 Organisation

The practical assignment consists of 14 x 2:45 h in the lab and next to that preparation at home. During this time each group (of max. 2 persons) must complete the practical assignments.

Chapters 2 – 4 of this document provide training assignments as introduction to the tools, and prepare parts for the later assignments. Chapters 6 to 8 are construction and design assignments where you are *encouraged* to come up with your own, creative, original solutions!

From Chapter 6 onwards, the difference between the available hardware platforms becomes significant.

#### 1.3.1 Assignments

This document contains multiple assignments, growing in own creativity/freedom and complexity.

Next to that, tips and pointers to further information are given in the text.

**Note** that Assignment 15 on Design Space Exploration is placed at the end (Section 8.2), but it appears beneficial to read that one earlier, and keep that assignment in mind while working on the other assignments.

### 1.3.2 Recording intermediate progress

When an assignment is finished, find a Teaching Assistant (TA) and show the TA that you finished the assignment and let them sign off the assignment on the list. So we are able to keep track on your progress and see whether things are going smoothly.

### 1.3.3 Lab location

The setups, computer boards and JIWI are available in RaM LabOne, Carré 3434. Each lab location has a unique JIWI. Once your group has chosen a setup to work with, you have always to use that associated location.

## 1.4 Responsibility and fraud; Using AI tools

All group members share responsibility for the work handed in. When you distribute work among group members, check each other's work, discuss the work among all group members, do you understand it all?

You must submit original work.

Plagiarism, i.e. copying of someone else's work without proper reference, is a serious offence which in all cases will be reported to the Examination Board. Refer to UT's Student Charter (2020).

In cases where a non-trivial amount of work is copied *with* proper reference, indicate which parts are copied and which parts are your own original work. The copied work will not be considered for grading.

### 1.4.1 Using ChatGPT and other AI tools

You are allowed to use ChatGPT and other AI tools to learn (just as using other sources like the internet). However, you should answer questions in your own words and produce programs using your own code. And, if you take code pieces / texts from the ChatGPT etc tool results, you need to add a reference / mention that you have taken this from that source, as is usual with referencing to literature and comparable sources.

Answers or code that are obviously copied from ChatGPT or other AI tools result in a score of 0 (zero) points. Excessive copying without proper referencing from ChatGPT / AI tools is considered plagiarism / fraud and is reported to the Exam Board.

Note that it is known that effortful learning (so, it costs some work to get at the solution) gets deeper and more durable / sustainable learning results and better insight. So, seriously working on the Assignments makes you better prepared to produce the final report and for the oral exam, and helps you retaining these skills and insight for later.

### 1.4.2 What constitutes fraud?

Material written by Arend Rensink is reused here.

When it comes down to handing in assignments, every year there are students who do not understand the borderline between, on the one hand, cooperating and discussing solutions between groups (which is allowed), and on the other, copying or sharing solutions (which is forbidden and counted as fraudulent behaviour). Here are some scenarios which may help in making this distinction.

**Scenario** Peter and Lisa are quite comfortable with programming and have pretty much finished the assignment. Mark and Wouter, on the other hand, are struggling and ask Lisa how she has solved it. Lisa, a friendly girl, shows her solution and takes them through it line by line. Mark and Wouter think they now understand and go off to create their own solution, based on what they saw. Is this allowed or not?

**Verdict** No problem here, everything is in the green. It is perfectly fine and allowed for Lisa to explain her solution, even very thoroughly. The important point is that in implementing it themselves and testing their own solution, Mark and Wouter are still forced to think about what is happening and will gain the required understanding, though probably they will not get as much out of it as Lisa (explaining stuff to others is about the best possible way to learn it better yourself!).

**Scenario** The start is as in the previous case. However, while Mark and Wouter implement their own solution, inspired by that of Lisa, some error crops up which they do not understand. Lisa has left by now; after they mail her, still trying to be helpful she sends them her solution for them to inspect. They inspect it so closely that in the end their solution is indistinguishable from Peter and Lisa's, except for the choice of some variable names and the comments they added themselves. Is this allowed or not?

**Verdict** This is now a case of fraud. Three are at fault: Lisa for enabling fraud by sending her files (even if it was meant as a friendly gesture) and Mark and Wouter for copying the code. Peter was not involved, developed his own solution (together with Lisa) and is innocent.

**Scenario** Alexandra and Nahuel are not finished, and the deadline is very close. The same holds for Simon and Jaco. On the Friday night train home, Jaco and Nahuel meet and during the 2-hour train ride work it out together. They type in the same solution and hand it in on behalf of their groups. Is this allowed or not?

**Verdict** This is also a case of fraud. Actually there are two problems here. The first is that both Nahuel and Jaco handed in code on behalf of their groups that had been developed by them alone, without their partners. This is unwise and against the spirit of the assignment (Alexandra and Simon also need to master this stuff!) but essentially undetectable and not fraudulent. The second problem is that the solution was developed, and shared, in collaboration between two groups; this is definitely forbidden. All four students are culpable; Alexandra and Simon cannot hide behind the fact that they did not partake in the collaboration, as they were apparently happy enough to have their name on the solutions and pretend they worked on it, too.

**Note** that we are not on a witch-hunt here: let us stress again that cooperating and discussing assignments is OK, even encouraged; it is at the point where you start copying or duplicating pieces of code that you cross the border.

## 1.5 Document History and Disclaimer

When you find a mistake or have remarks about this document, you should contact one of the TAs or the teacher.

As we also continuously improve this document, newer versions appear regularly, see its date. The latest version can be found on Canvas; Be sure to always use the latest version of this guide.

This document has seen quite some versions over the years. Contributors to this document, the underlying software and electronics, next to the authors, are: Marcel Schwirtz, Bas de Quinze, Douwe de Vries, Peter Kingma, Hidde Hoorweg, Bram Wesselink, Rienk van der Wijk, Floriaan Bulten, Florian Hagens, Steven Wijnja, Jos Vos, Maarten Bezemer, René Moll, Jan Jaap Kempenaar, Jos van 't Hof, Mickey Derks, Rinse Wester, Arvid van den Brink, Koen Zandberg, Wolfgang Baumgartner, Silke Hofstra, Hans de Jong. Marjon Kuipers supported on the hardware.

Special thanks to Arend Rensink for the Subsection "What constitutes fraud?".

## 2 Getting familiar with the tools

In this chapter, the main objective is to get familiar with the tools. First, a short walkthrough is given for creating a working FPGA setup using the two tool suites: for the ICE40 and the Cyclone 5. After that, a quick demonstration of the ARM tools is performed.

All the necessary files can be found on the RAM Git, in the laboratory-files repository. To clone this repository, run the following command in your terminal:

- `git clone https://git.ram.eemcs.utwente.nl/repository-esl/laboratory-files`

### 2.1 Raspberry Pi and icoBoard Tools

In this Section, the tool suite used for the Raspberry Pi 4 and icoBoard FPGA HAT (Hardware Attached on Top) are introduced.

The tool suite consists of:

- Visual Studio Code, often abbreviated as VS Code.
- Yosys: Verilog synthesis tool.
- Nextpnr: Place and Route tool for the Lattice FPGA on the icoBoard.
- gcc: C compiler.
- IcoProg: Executable to program the Lattice FPGA with the generated binary.

This chapter guides you through the process of setting up both your machine and the Raspberry Pi 4 – icoBoard combination. The workflow using the tool suite is shown in Figure 2.1. Note that yosys and nextpnr can run on any Linux machine, so also on a Linux virtual machine on your Windows or Mac laptop.

#### 2.1.1 Goals

- Get familiar with the C compiler and yosys and nextpnr combination for hardware design on the remote Raspberry Pi 4 environment.

#### 2.1.2 Set up the Raspberry Pi, including the ICE40 FPGA tool chain

To set up the Raspberry Pi, you will be provided with an empty SD-card. Use the Raspberry Pi Imager (see links) to install Ubuntu 24.04.04 LTS Server (64-bit ARM), the image can be downloaded directly from the raspberry pi imager (24.04.04 LTS).

Please setup your username and password in the setup, the network ip can be found on the baseplate of the raspberry pi.

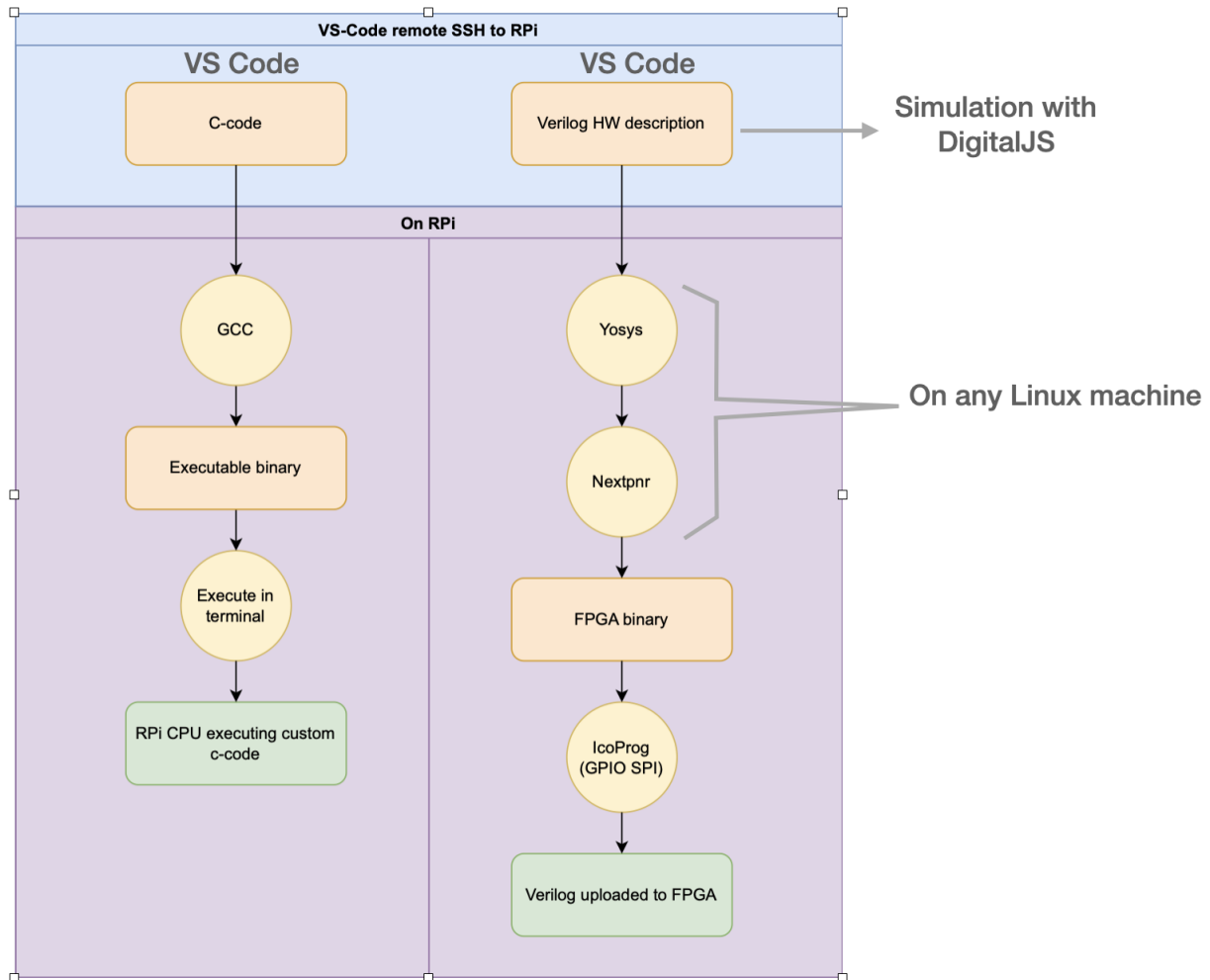
After the SD card is flashed, use SSH or VSCode to connect to the raspberry pi, (Note, when the PI is connected through the *GREEN* cable, it is *NOT* accessible via eduroam)

After connected to the pi, please execute the following commands to install the required packages and tooling.

Please execute the following commands in the terminal in the raspberry pi in your chosen work directory.

These following commands are ALSO available in the following git in the HandyScripts folder [Laboratory folder](#) allowing you to CTRL+C and CTRL+V.

- `sudo apt update`  
This updates the package manager.



**Figure 2.1:** Tool-chain overview for Raspberry Pi 4 and icoBoard development. Instead of using DigitalJS, you can use iverilog and gtkwave to simulate verilog modules, see Appendix B.

- `sudo apt install yosys nextpnr-ice40-qt libboost-all-dev libeigen3-dev qtcreator qtbase5-dev qt5-qmake fpga-icestorm g++ raspi-config`  
This installs the following required packages:
  - yosys  
Hardware synthesis tool
  - nextpnr-ice40-qt  
Place and Route tool
  - libboost-all-dev libeigen3-dev qtcreator qtbase5-dev qt5-qmake  
Note: libboost-all-dev appears to give errors, but if the system still compiles this can be ignored. Dependencies for graphical view
  - fpga-icestorm  
Tool to convert place and route output to bitstream for fpga
  - g++  
C++ compiler, also used for IcoProg
  - raspi-config  
To disable hardware from claiming needed GPIO pins
- `sudo apt install git libgpiod-dev gpiod`
- `git clone https://git.ram.eemcs.utwente.nl/repository-esl/icoprogram.git`  
to clone IcoProg source code.

- `g++ src/icoprogram.cpp src/gpio_interface.cpp -o icoprogram -lgpiodcxx`  
to compile and obtain icoprogram executable.

Now we need to disable the kernel from loading GPIO modules, as we need access to the those GPIO modules ourselves. This is done through `sudo raspi-config`.

In this menu, go to Interface Options using the arrow keys and enter to confirm. Here, I2C, SPI, Serial and 1-wire need to be disabled. Press Enter on each of the options, followed by entering No, alternatively `sudo dtparam spi=on/off` can be used.

Afterwards, use the right arrow key to enter Finish. You should now reboot your Raspberry Pi.

If all went well, the executable binary called icoprogram can now be used to upload an FPGA bitstream to the IcoBoard, which we will do later on.

- [digitalJS Verilog Simulator](#)
- [Verilog-HDL for Verilog formatter and linter](#)

You can use the Extensions tab to find and install these extensions, or install them in VS Code by having VS Code open, pressing Ctrl+P and running the following 2 commands sequentially:

---

```
ext install mshr-h.VerilogHDL
ext install yuyichao.digitaljs
```

---

### 2.1.3 Optional: Set up the ICE40 FPGA tool chain on your own machine

When working on the Raspberry Pi, you can also use the tools on your own machine, for example to have a more comfortable development experience and possibility to prepare for the labs beforehand. In case you do *not* have a laptop running Linux, you can install a virtual machine. See the page on Canvas on installing a virtual machine. One way is by installing VirtualBox and then installing Ubuntu 24.04.

To install the tool chain on Linux or the Linux VM, run:

- `sudo apt update`  
This updates the package manager.
- `sudo apt install yosys nextpnr-ice40-qt libboost-all-dev libeigen3-dev qtcreator qtbase5-dev qt5-qmake fpga-icestorm g++ raspi-config`  
This installs the following required packages:
  - yosys  
Hardware synthesis tool
  - nextpnr-ice40-qt  
Place and Route tool
  - libboost-all-dev libeigen3-dev qtcreator qtbase5-dev qt5-qmake  
Dependencies for graphical view
  - fpga-icestorm  
Tool to convert place and route output to bitstream for fpga
  - g++  
C++ compiler, also used for IcoProg
  - raspi-config  
To disable hardware from claiming needed GPIO pins

### 2.1.4 Blinking LED

Now that all the necessary tools are installed, we can write a little Verilog file to get the LED blinking on the icoBoard. Note that this tutorial can be followed using VS Code remotely on the Raspberry Pi, and/or on your VM.

To do this, create a project folder, Blinky is a good name, and open this folder in Visual Studio Code using File --> Open Folder. Then, create a TopEntity.v file using File --> New File and copy the following code into it. This code, along with the necessary .pcf file, is available in the starter kit in the RAM gitlab.

---

```

module TopEntity(
    input clk,
    output reg led1 = 0
);

reg[31:0] count = 0;

always @(posedge clk) begin
    if (count >= 99999999) begin           // Time is up
        count <= 0;                       // Reset count register
        led1 <= ~led1;                   // Toggle led (in each second)
    end else begin
        count <= count + 1;               // Counts 100 MHz clock
    end
end
endmodule

```

---

(To simulate this in DigitalJS or another simulator, change the operand in the if-statement to a lower value, such as 5.)

Now, to synthesize this Verilog design, open a terminal using Terminal --> New Terminal. In the terminal, execute the following yosys and nextpnr commands (note the double dashes in the second line):

---

```

yosys -p 'synth_ice40 -top TopEntity -json ice40.json' TopEntity.v
nextpnr-ice40 --hx8k --json ice40.json --pcf ico-jiwy.pcf --asc
    ice40.asc
icepack ice40.asc ice40.bin

```

---

Here we instruct yosys to synthesize for an ice40 FPGA, of which the icoBoard is one. And tell it the top entity to synthesize is called TopEntity and to output the result in ice40.json. The nextpnr command tells nextpnr to place-and-route specifically for the ice40 hx8k FPGA, with a json input from yosys in ice40.json, and to output the placed-and-routed design into ice40.asc. Finally, the icepack command converts the placed-and-routed design to the corresponding FPGA bitstream to be uploaded to the FPGA using IcoProg on the Raspberry Pi.

This means that if you have been following along on your VM/machine, now is the time to copy the ice40.bin file to the Raspberry Pi. If you are already working on the Raspberry Pi, it is already there, of course.

So let us upload the bitstream to the FPGA.

Open a terminal on the Raspberry Pi, suggestions are either via a remote VS Code session, or SSH to the Raspberry Pi. With a terminal open and in the same location as the ice40.bin file, run:

---

```

./icoprogram -R
./icoprogram -p < ice40.bin

```

---

The -R resets the FPGA, and -p uploads the input that follows, so the ice40.bin is redirected using the < operator to function as input.

If all went well, the LED on the IcoBoard should be flashing in a regular pattern! Now add more LEDs and make them blink in any order.

**Required hardware: Raspberry Pi**

**Assignment 1:** *Congratulations! You now have a functional IcoBoard & Raspberry Pi combination. Show this, including the multiple blinking LEDs, to the TA for sign-off.*

**2.1.5 Links**

- Raspberry Pi Imager: <https://www.raspberrypi.com/software/>
- Ubuntu Server (24.04.04): <https://ubuntu.com/download/raspberry-pi>
- Visual Studio Code: <https://code.visualstudio.com/>
- IcoBoard documentation: <http://icoboard.org/about-icoboard.html>
- Yosys: <https://github.com/YosysHQ/yosys>
- Nextpnr: <https://github.com/YosysHQ/nextpnr>
- IceStorm: <https://github.com/YosysHQ/icestorm>
- IcoProg: <https://git.ram.eemcs.utwente.nl/repository-esl/icoprogram.git>

**2.2 Altera tools****2.2.1 Goals**

- Get familiar with the Altera Toolchain and the ARM Hard Processor System (Figure 2.2).

**2.2.2 Altera Quartus & Qsys****Installing the Altera tools**

For the Altera tools, you can either use the `xoc2.ewi.utwente.nl` server, or install the Quartus software on your own machine.

To use the xoc2 server, follow the following steps:

1. Install the X2Go client (see the Canvas page for instructions).
2. Make a new session with host `xoc2.ewi.utwente.nl`, set session type to XFCE, and use your s-number as login.
3. If you are outside the UT network, use EduVPN.
4. Once logged in, launch a terminal and enter the following commands:

---

```
module load intel/quartus/std/23.1.1
quartus
```

---

If you install it locally: do not install the newest version, but version 23.1. This is because some tooling is missing in the newest version.

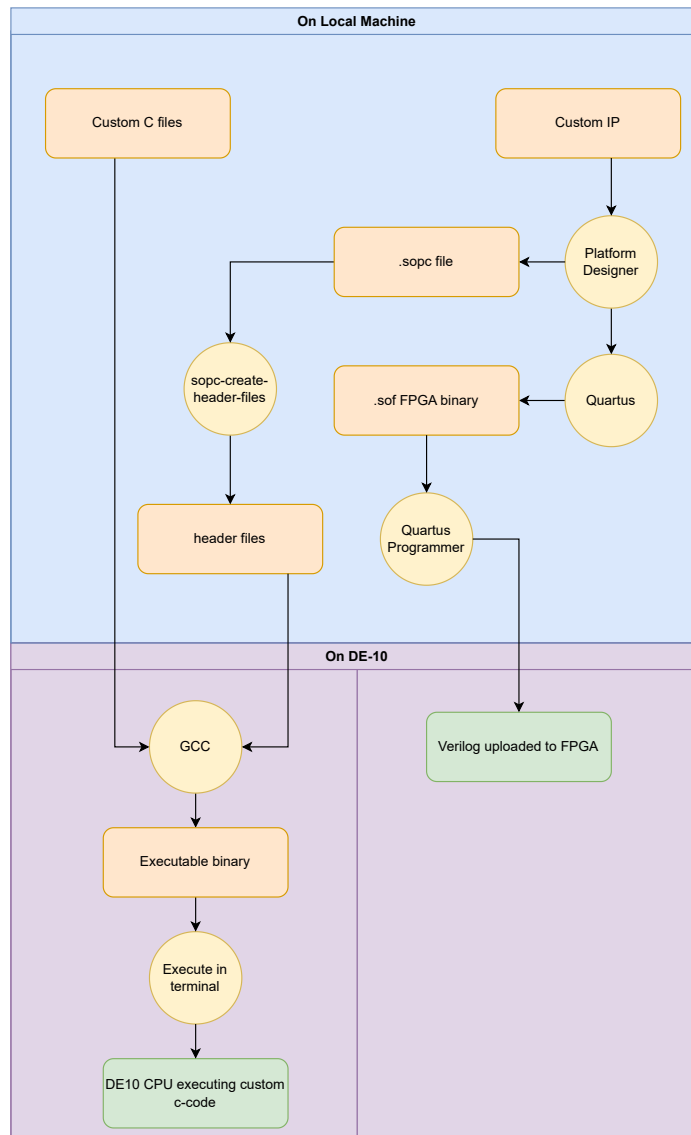
**Blinking LED example**

To get familiar with the Altera tools, we start by uploading a blinking LED project to the FPGA. First we have to make sure the DE10 is able to boot. This is done by flashing the SD card with the image provided on Canvas. The flashing can be done using programs like `dd`, `Etcher` and `Rufus`.

There are two ways of connecting to the DE10, namely `ssh` and `serial`. The default login is `root` with password `EmbeddedSystemsLab`. To change the password run the command `passwd`.

**Serial**

The baud rate of the DE10 is 115200.



**Figure 2.2:** Tool-chain overview for FPGA/ARM development.

**Linux/Mac** Possible programs are `screen` and `minicom`.

**Windows** Use the program `PuTTY`.

## SSH

To connect to the DE10, run the command `ssh root@<ip-address>`. You can find out the ip address of the DE10 by first connecting by serial and then running `ip a`.

Now that we are able to log into the DE10, we are ready to make the first FPGA design.

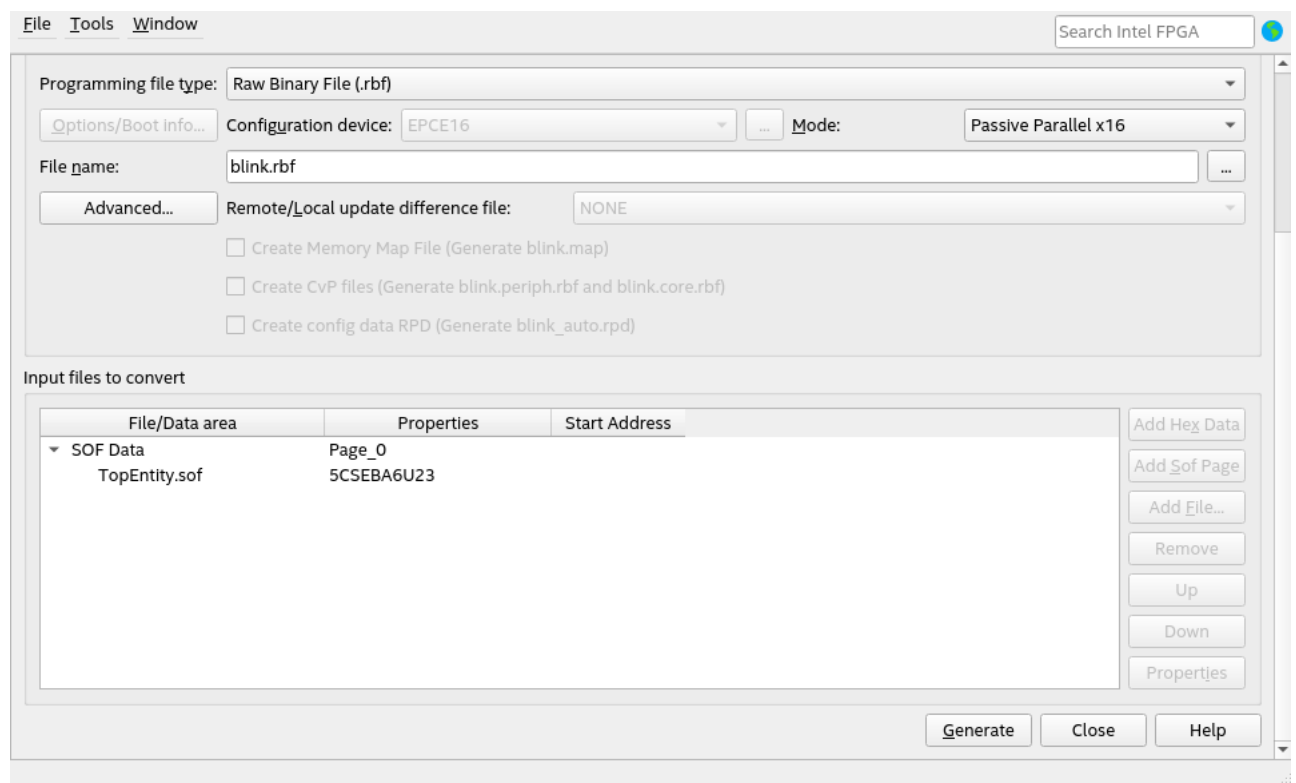
- Open a new project in Quartus
- Choose a name for the project and top level entity, for example `blink` and `TopEntity`.
- Choose empty project.
- Add the verilog design file of the blink example.
- Select the `5CSEBA6U23I7`.
- Select nothing with the EDA Tool settings.
- Check the selected values and Finish the project setup
- Under `Assignments` --> `Import Assignments` import the `DE10_NANO_PINOUT.csv`

- Compile the program.
- Select File --> Convert Programming Files
- Make sure the settings are identical to those shown in Figure 2.3.
- Select generate.
- Connect to the DE10 using ssh: `ssh root@<ip-address>`
- Retrieve the .rbf file from the xoc2 server to the DE10 by scp (from DE10 terminal):  

```
scp s<student-number>@xoc2.ewi.utwente.nl:<directory>/<filename>
<directory on DE10>
```
- Make a directory `mkdir -p fat`
- Mount the partition `mount /dev/mmcbk0p1 fat`
- Copy the Raw Binary File. The bootloader expects the file to be called `soc_system.rbf`, so:  

```
cp <filename> fat/soc_system.rbf
```
- Unmount the partition by: `umount fat`
- Reboot the DE10 by: `reboot`

Now the DE10 will program the FPGA on every reboot.



**Figure 2.3:** Settings to convert the output file to rbf

### Required hardware: Altera DE10-NANO

**Assignment 2:** Follow and complete the tutorial, when you are done inform a TA to check off your work.

### 3 Introduction to the logic analyser

To be able to analyse any digital signals, a logic analyser is used during this lab. This can come in handy when either the JIWIY does not do what is expected, or when the communication between the Raspberry Pi and the icoBoard is not yet working.

#### 3.1 Goals

- Getting comfortable with PulseView and the Logic Analyser.

#### 3.2 Tools

- PulseView: To install PulseView, follow the installation instructions for your platform at <https://sigrok.org/doc/pulseview/unstable/manual.html#installation>.

#### 3.3 Assignment

To get started with PulseView, follow these steps:

1. Open the just installed PulseView, the window should look like Figure 3.1.
2. Connect the Logic Analyser via USB to the pc/laptop with PulseView installed.
3. Open the `Connect to device` window using the drop-down menu next to the demo-device text in the menu.
4. In this window, in Step 1, select `fx2lafw` as driver.
5. In step 2, select `USB` as the interface, and click on `Scan for devices...` in step 3.
6. Select `Saleae Logic` with `8 channels` in step 4.
7. The window should look like Figure 3.2, if so, press `OK` at the bottom of the window.
8. You should now be able to see the 8 channels of the Logic Analyser in PulseView.

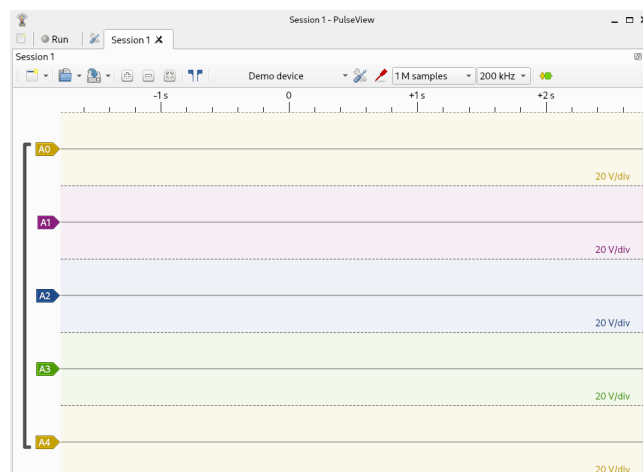
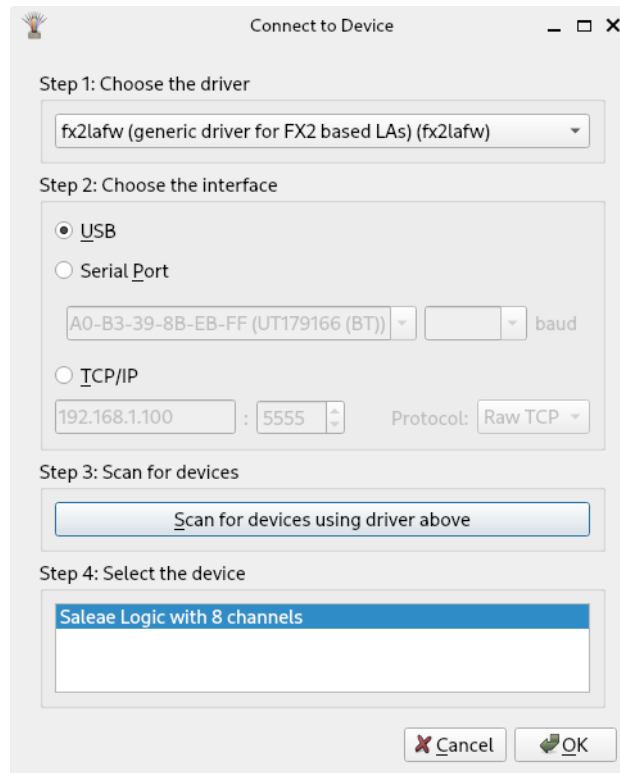


Figure 3.1: PulseView's opening window

Let us analyse some "logic", and use the Chip Enable 0 (CE0) from the SPI connection on the Raspberry Pi's GPIO as generic IO. It is available on the breakout board for probing with the Logic Analyzer.

**ONLY** connect probes to the circular probe terminals on the SPI breakout board and/or the probe terminals on the PMOD measurement PCB.

**DO NOT** stick probes on any other place, as they do **NOT** belong there !!



**Figure 3.2:** PulseView's "connect device" window

Copy the following code to a file on the Raspberry Pi, name it `main.c`, and compile it using: `gcc main.c -lgpio -o main`. Then, run the executable using `./main`. This code should alternate the CE0 pin between HIGH and LOW.

```
#include <gpio.h>
#include <stdio.h>
#include <unistd.h>

int main(int argc, char **argv) {

    char *chipname = "gpiochip0";
    unsigned int line_num = 8;
    unsigned int val;
    struct gpiochip *chip;
    struct gpiochip_line *line;
    int i, ret;

    chip = gpiochip_open_by_name(chipname);
    if (!chip) {
        perror("Open chip failed\n");
        goto end;
    }

    line = gpiochip_get_line(chip, line_num);
    if (!line) {
        perror("Get line failed\n");
        goto close_chip;
    }

    ret = gpiochip_line_request_output(line, "user", 0);
```

```
if (ret < 0) {
    perror("Request line as output failed\n");
    goto release_line;
}

val = 0;
while (true) {
    ret = gpiod_line_set_value(line, val);
    if (ret < 0) {
        perror("Set line output failed\n");
        goto release_line;
    }
    printf("Output %u on line #%u\n", val, line_num);
    sleep(1);
    val = !val;
}

release_line:
    gpiod_line_release(line);
close_chip:
    gpiod_chip_close(chip);
end:
    return 0;
}
```

---

Now, connect the Logic Analyser's GND to the SPI breakout board's GND, and similarly Channel 0 to CE0. And finally, press the run button in the top left of PulseView to start a signal capture.

### Required hardware: Raspberry Pi

**Assignment 3:** *You should now be able to verify this rudimentary signal output in PulseView. In a later section, we will look at analysing communication protocols (like SPI) specifically.*

The rest of the Sigrok PulseView manual holds more information about its functionality in general, especially when you do not have a lot of experience with wave viewers or signal analysers, it is worth a read!

#### 3.3.1 Links

- Sigrok/PulseView installation:  
<https://sigrok.org/doc/pulseview/unstable/manual.html#installation>

## 4 Building-blocks tutorial

This chapter gives an introduction on how to develop user code for both the FPGA and the ARM system. For the FPGA an encoder and PWM module have to be made. For the ARM system, the GStreamer framework must be used to grab data from the webcam and process it.

Assignments in this chapter and Chapter 6 provide knowledge for Assignment 15, the Design-Space Exploration of the final demonstrator. It is beneficial to keep Assignment 15 in mind, while working on the building blocks.

### 4.1 Raspberry Pi and icoBoard

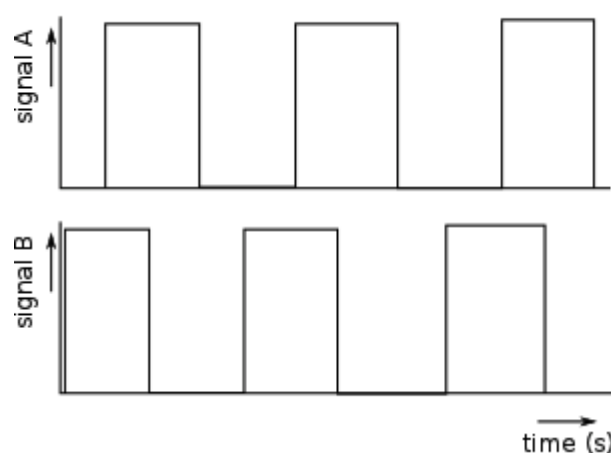
#### 4.1.1 Quadrature encoder in Verilog

**Required hardware: None**

**Assignment 4:** *Design, develop, and test a Verilog design for reading a quadrature encoder. When the module is done, show it to one of the TAs.*

#### Tips and Remarks

- Look up how an encoder works.
- Check the PCB connections and icoBoard pinout in Appendix A.
- An input signal may look like Figure 4.1. Depending on which signal arrives first (phase shifted 90 degrees), the direction can be determined.
- The encoders usually generate about 500 to 2000 pulses for one rotation. The setups have different encoders, so a different number of pulses per rotation. You can check this after the next assignment.
- Determine relevant parameters, settings, status states, and such. Which are relevant at design time, and which are during run time? Document this for later use.
- Test your module properly, using test benches. See Appendix Appendix B for more information on how to do this and which tooling to use.



**Figure 4.1:** Example signal form an encoder

### 4.2 ARM – the one on the Raspberry Pi

This section focusses on getting image data from the camera. To get data from the camera, the GStreamer framework is used. This framework can either be used from the command line, or

in a program. First we build a basic pipeline to get video data from the webcam to a file. Later we extend this to get the video data with a C program. There are a few links at the end of the section pointing to the software used to accomplish this.

#### 4.2.1 Basic GStreamer pipeline

If you are not familiar with GStreamer, it is good to read about it on its website and play with the pipelines on your own computer. If the concepts are clear, it is much easier to understand what is going on in this chapter.

To see if we can get any video from the webcam, first we make a command-line streamer pipeline.

##### Vision source

The first part in the pipeline is the source, there are a few different setups. The cameras currently there are similar to one of the following two options:

- Logitech c250 webcam. In the RAM gitlab, there is a file called `logitech_uvc_frame_format_list.pdf` that shows what kind of output formats are supported by this device.
- Waveshare OV2710 USB camera. This currently only supports 1920x1080 at 6 frames a second, but can use both MJPEG and YUY2.

The logitech webcam can produce MJPEG and YUV output in various different formats. The waveshare camera has access to MPJEG and YUY2, of which the second is a lower bit version of YUV. Try to understand what the difference is between these ways of storing images before you continue with the tutorial. Also make sure you know which camera you have.

To use the webcam in our GStreamer pipeline, we need the `video4linux2` plug-in. You can check if it is present by using the `gst-inspect` command:

```
gst-inspect-1.0 v4l2src
```

This should show the capabilities of the plugin. If it is not installed you can get it by typing in the correct commands in the Raspberry Pi/DE10 terminal:

---

```
sudo apt-get install libgstreamer1.0-dev gstreamer1.0-plugins-good  
gstreamer1.0-tools
```

---

##### Testing the first pipeline

Now that we know that the source is present, we can test it with the following command:

```
gst-launch-1.0 -v -e v4l2src device=/dev/video7 ! fakesink
```

The modifier `-v` ensures verbose mode, showing more information than normal and `-e` ensures a clean pipeline shutdown after a `ctrl-c`. The device argument for the `v4l2src` tells it what video device it needs to use, which is in this case `/dev/video7`. You can check the `/dev` folder to figure out which video device to use:

```
ls /dev/video*
```

The output of the source is connected to the `fakesink` GStreamer component. This receives information and throws it away, not very handy in real life, but really useful for testing. In this case, we do not specify what kind of video format should be outputted from the webcam. Now, GStreamer will automatically choose one of the output formats and we do not know what. We can ensure the type of output format by setting a `capsfilter`. The next command forces the webcam to output a MJPEG picture of 160x120 at 30 frames per second, including an explicit `encoding-to-jpeg` command (some of our webcams need this explicit encoding command) – put it on *one* line:

---

```
gst-launch-1.0 -v -e v4l2src device=/dev/video7 ! jpegenc !  
image/jpeg,width=160,height=120,framerate=30/1 ! fakesink
```

---

**Tip:** Try to find a way to get the YUV output from the camera.

The waveshare camera can *not* be limited to this format, and this is why you can instead use the command below. This command similarly forces the output to a MJPEG picture of 1920x1080 at 6 frames per second – put it on *one* line:

---

```
gst-launch-1.0 -v -e v4l2src device=/dev/video7 ! jpegenc !
    image/jpeg,width=1920,height=1080,framerate=6/1 ! fakesink
```

---

### Movie

Now that we have our source up and running, we can store these images into a movie. The MJPEG images can be stored easily in an avi file using the avimux plugin. The pipeline looks as follows – put it on *one* line:

---

```
gst-launch-1.0 -v -e v4l2src device=/dev/video7 ! jpegenc !
    image/jpeg,width=160,height=120,framerate=30/1 ! avimux !
    filesink location=file.avi
```

---

This video can then be transferred to the computer using scp and be viewed using a viewer like vlc.

### Decoding MJPEG

In the final assignment you might want to do some image processing to get information out of the video stream, which is not handy to do on JPEG data. To decode this data we can use the JPEG plugin from GStreamer. This plugin can decode the images to YUV data so that it can be processed by your image processing algorithm. Maybe you can find a way to convert them to RGB images if you want to use RGB images. To use the JPEG plugin execute the following command – put it all on *one* line:

---

```
gst-launch-1.0 -v -e v4l2src device=/dev/video7 ! jpegenc !
    image/jpeg,width=160,height=120,framerate=30/1 ! jpegdec !
    filesink location=file.yuv
```

---

These files can be played with yuv players, for mac there is one called GLYUVplay. Linux users can use MPlayer for this task with the following command line command, when using the waveshare camera the fps should also be specified:

```
mplayer -demuxer rawvideo -rawvideo w=160:h=120:format=i420 file.yuv
```

Other players are available on the internet.

### 4.2.2 Building a C application

In the previous section, you have learned how to grab the camera images from the webcam and put them into files. However, we want to build the pipeline in a program. The first thing to do is grab the source from [GStreamer's Hello World program](#) and examine it.

#### Required hardware: Raspberry Pi or Altera DE10-NANO

**Assignment 5:** *Figure out how the application works, and then convert it to implement the yuv storing pipeline. Show it to the TAs to check off this assignment.*

#### Building for the development PC

To compile GStreamer applications for the development PC you can use the following command:

```
gcc hello.c -o gst_hello $(pkg-config --cflags --libs gstreamer-1.0)
```

### 4.2.3 Getting the data

For your final assignment, you need to get to the data which is in the pipeline. There are several ways to do this. In this guide, the appsink GStreamer plugin is used, use [this link](#).

You can replace the filesink you used previously with the appsink and connect callback functions to this element. Show that you can get the data from the pipeline and print out some information about it, like average pixel value or something similar.

#### Required hardware: Raspberry Pi or Altera DE10-NANO

**Assignment 6:** *Implement the appsink element and show the TAs that it works.*

To compile this, your compiler will probably complain that it cannot find the appsink library. In that case, you can modify the compile commands from the previous section like this:

```
gcc hello.c -o gst_hello $(pkg-config --cflags --libs gstreamer-1.0 gstreamer-app-1.0)
```

### 4.2.4 Links

- GStreamer framework mainpage:  
<https://gstreamer.freedesktop.org/>
- GStreamer appsink manual page:  
<https://gstreamer.freedesktop.org/data/doc/gstreamer/head/gst-plugins-base/libs/html/gstreamer-app.html>
- Example for appsink and some more related things:  
<https://gstreamer.freedesktop.org/documentation/tutorials/basic/short-cutting-the-pipeline.html?gi-language=c>
- Two other examples for appsink:  
<https://github.com/GStreamer/gst-plugins-base/blob/master/tests/examples/app/appsink-src2.c>  
<https://searchcode.com/file/63275229/gst-plugins-base-0.10.28/tests/examples/app/appsink-src.c>
- MJPEG wikipage  
[https://en.wikipedia.org/wiki/Motion\\_JPEG](https://en.wikipedia.org/wiki/Motion_JPEG)
- YUV colorspace wikipage  
<https://en.wikipedia.org/wiki/Yuv>
- GStreamer capsfilter  
<https://gstreamer.freedesktop.org/data/doc/gstreamer/head/gstreamer-plugins/html/gstreamer-plugins-capsfilter.html>

## 4.3 Cyclone 5 on DE10-Nano

### 4.3.1 Goals

- Build you own quadrature encoder and PWM module.
- Incorporate custom modules to the SOPC builder.
- Control hardware logic from software.

### 4.3.2 Create a demonstrator project

Before continuing with the assignments, we first create a demonstrator project we can use to demonstrate the quadrature encoder and PWM module.

1. Open Quartus and click New Project Wizard.
2. At the Introduction screen click Next.
3. Choose a location to store your project. Call your project and top level entity:  
DE10\_NANO\_SoC  
Click Next.

4. Browse [...] and add the DE10\_NANO\_SoC.v to the project. Click Next.
5. Select the Device Family and Device:
  - Family: Cyclone V
  - Device: 5CSEBA6U23I7
6. Click Finish

### ARM system integration

The next steps will guide you through the process of connecting an IP to the ARM processor via the Avalon bus. To do so, some logic is required to glue the IP to the processor, therefore, a wrapper is made. This wrapper implements a very simple Avalon memory mapped interface for bus communication, see for more information the [Avalon interface specifications](#).

**Note:** By default the bus width is set to 32 bits, but may be reduced to 16 or even 8 bits. While this may be useful in some cases, it does require addition logic.

- Go get the labartory files for this assignment from the esl repo: [Laboratory files](#).
- The folder contains two folders, one with C code and one with Verilog.
- In the project directory, create a new folder named ip and move the folder ESL\_bus\_demo into this folder. Platform Designer uses a number of paths to search for IPs, one of these is the local ip folder.
- In the project directory, move the soc\_system.qsys file into this folder.
- Open Platform designer, and open the soc\_system.qsys.
- In the component library, left plane, under the project tree a section *Templates* should be present.
- Add the esl\_bus\_demo component, keep the default settings. Make sure the compon-ent is set up the same as seen in Figure 4.2.
- Generate the HDL of the system, and add the .qip file to your Quartus project.
- Import the csv for the pin assignment, similar to Assignment 2.
- Connect the LEDs in the DE10\_NANO\_SoC.v file and compile.
- If you get errors at the Fitter stage with Output buffer atom ..., open the Tcl Scripts window, and run hps\_sdram\_p0\_parameters.tcl and hps\_sdram\_p0\_pin\_assignments.tcl.
- Generate the rbf and move it to the DE10, similar to Assignment 2.
- Move the file socfpga\_cyclone5\_de0\_nano\_soc.dtb to the same directory as the rbf file and reboot.
- **Note:** This only needs to be done once.
- Move the C code to the DE10 and compile it.
- Notice the use of the definition HPS\_0\_ARM\_A9\_0\_ESL\_BUS\_DEMO\_0\_BASE and HPS\_0\_ARM\_A9\_0\_ESL\_BUS\_DEMO\_0\_SPAN. This is a reference, defined in soc\_system.h.
- To generate these headers for other systems use the command socp-create-header-files.
- **Note:** The command console is available in platform designer under Tools and then NIOS II command shell.
- Run the demo and watch the LEDs count down.

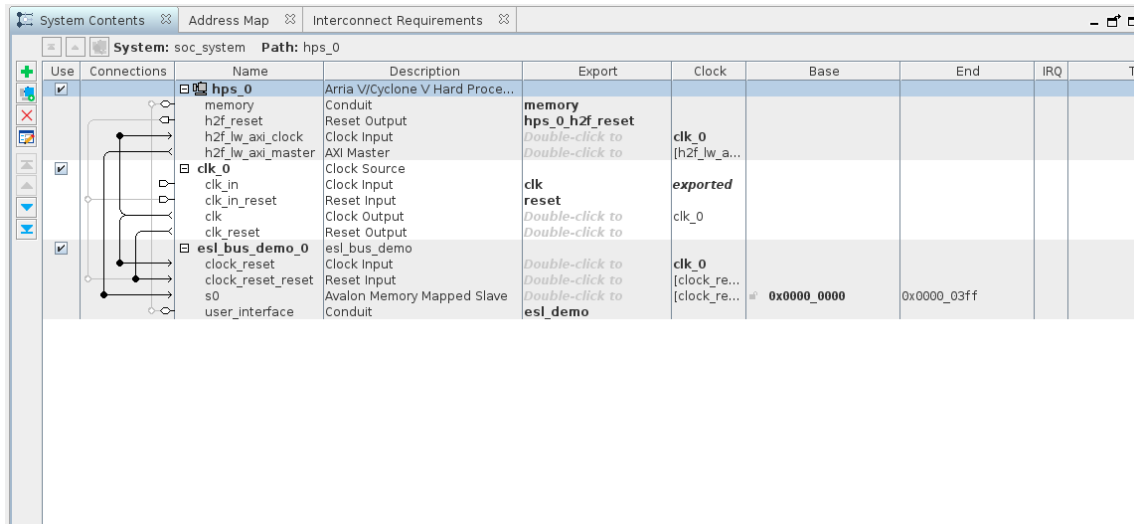
### 4.3.3 Quadrature encoder on the DE10-NANO

#### Required hardware: Altera DE10-NANO

**Assignment 7:** Now that you are familiar with adding an IP to a ARM system, add the encoder IP to the ARM system. Do not connect the FPGA to the JIWIY at this point, first show a TA your work.

#### Tips and Remarks

- Use the SWITCHES and LEDS on the FPGA, also provide the counter value in the terminal



**Figure 4.2:** Diagram of the system with custom logic connected to the Avalon bus

- Think of a way to demonstrate that the encoder is working. First without using an actual encoder, use the peripherals available on the DE10-NANO. If it works connect the encoder.
- Have a look at the files provided to you. These are documented and may give you an idea how to integrate it with your own project.
  - `esl_bus_demo.v` is the glue between the ARM system via the Avalon bus and the IP. Here the component is initialised, and any port maps are made.
  - `esl_bus_demo_example.v` is the actual IP.
  - `esl_bus_demo_hw.tcl` a Tcl script which is executed by Platform Designer to configure and integrate the component.
- Think of how you will communicate with the processor via the Avalon bus. The SOPC builder needs to know some information about your implementation. See the interface properties in the Tcl file.

#### Required hardware: Altera DE10-NANO

**Assignment 8:** Connect your IP to the setup and find out the properties of the encoders.

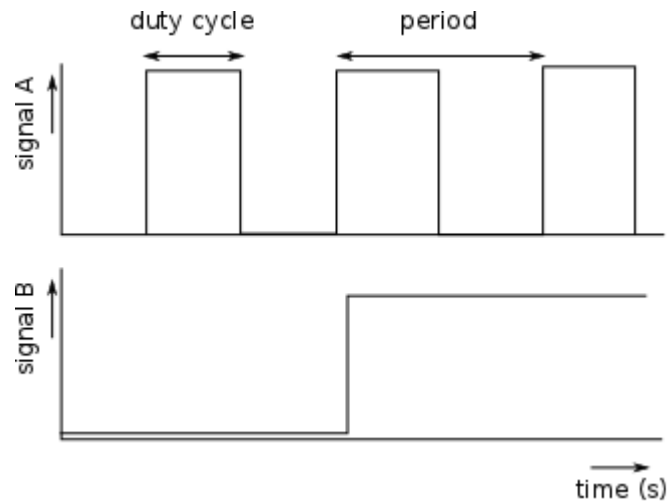
#### 4.3.4 Pulse width modulation

#### Required hardware: Altera DE10-NANO

**Assignment 9:** Design and develop a PWM module and show a TA that the module is functioning correctly.

#### Tips and Remarks

- Perform the same tasks as before, however now for a PWM module.
- The PWM module has to provide three output signals:
  - Signal INA and INB: two directional signals
  - Signal C: a PWM signal
- An explanation of how an H-bridge works can be found in Appendix C.
- Again find, determine and document any relevant parameters. Think of speed, enable/disable and frequency.
- Do pick a frequency outside the of the range humans can hear! (20[kHz] is a good pick).
- You can use an oscilloscope to verify your module.



**Figure 4.3:** Example signal for a PWM, the y-axes are not related to the module signal names.

#### 4.3.5 Links

NOTE: These some additional links might be added at a later date.

- Avalon interface  
[https://www.altera.com/literature/manual/mnl\\_avalon\\_spec.pdf](https://www.altera.com/literature/manual/mnl_avalon_spec.pdf)
- Embedded design handbook  
[https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/hb/nios2/edh\\_ed\\_handbook.pdf](https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/hb/nios2/edh_ed_handbook.pdf)
- SOPC Builder user guide  
[https://www.altera.com/literature/ug/ug\\_sopc\\_builder.pdf](https://www.altera.com/literature/ug/ug_sopc_builder.pdf)

## 5 Debugging SPI with the Logic Analyzer

While being able to verify the functionality of a GPIO pin going high and low logically every second is a good starting point, we are now going to expand upon this. In this Chapter you will implement SPI to communicate between the FPGA and the CPU. To be able to debug this more thoroughly, we will introduce the Logic Analyser from Chapter 3 as an SPI interpreter, to be able to debug possible communication problems in the future.

### 5.1 Goals

- Using PulseView and the Logic Analyser to view and verify the functionality of an SPI interface.

### 5.2 Tools

- PulseView

### 5.3 Assignment

To get started with this assignment, do the following steps:

- Connect the Logic Analyzer's probes to the SPI breakout board on the Raspberry Pi and remember which channel is connected to which pin. *Do not forget Ground (GND)!*
- Open PulseView and you should be greeted with a similar window as in Figure 3.1.
- Click on the Add protocol button to the right of the sample number and sample frequency, it looks like a yellow and green waveform.
- In the window that opens, search for `SPI`. You should find a plain SPI decoder, double-click on it. An SPI row should now be shown at the bottom of the main waveform window.
- Click on the SPI row's label, which is to the left. This should open a prompt where you can assign channels to each of the SPI channels, use your connections from Step 1.
- Compile the `main.c` of this chapter on the Raspberry Pi using `gcc -o main main.c`
- Set PulseView to capture `1 M samples` at `1 MHz` (samples per second).
- Start a capture in PulseView, and at the same time run the `main` executable. You could set a trigger in PulseView to start a capture when Chip Select goes low for ease of use.

You should now see an SPI capture in your waveform viewer, with an increasing number send each time on the PICO, and zeros on the POCL. This is because there is no SPI logic on the FPGA to return something, so let us add that.

- Use the `TopEntity.v` for this chapter, and synthesize it for the FPGA using steps indicated previously.
- Upload the bitstream using `IcoProg`.
- Start a new capture in PulseView, and run the `main` executable again.

Now there also should be a response with increasing numbers from the FPGA (so on the POCL), as it will reply with the data last sent! These are logically one SPI exchange later, as SPI is a full-duplex communication protocol.

#### Required hardware: Raspberry Pi

**Assignment 10:** *Congratulations! You should now be able to debug your communication protocol between the Raspberry Pi and icoBoard, and/or able to debug any signal send to the JIWI!*

## 6 Combining the hardware

### 6.1 Goals

- Develop a communication method to transport data between the Raspberry Pi and the icoBoard, suitable for the final assignment.

As discussed in Chapter 1, there are two different hardware implementations. The combination of an ARM core and FPGA on the Altera DE10-NANO and a Raspberry Pi with an icoBoard with FPGA. So, the platforms you used during the development in the training chapters.

For the DE10, you already have achieved communication between the ARM core and the FPGA in the previous chapter, so the focus here is on the Raspberry Pi and icoBoard combination.

The Raspberry Pi and icoBoard are connected via the 40-pin GPIO connector on the Raspberry Pi, of which `SPI01` on the Raspberry Pi is connected to the icoBoard's FPGA.

### 6.2 Assignments

In the RAM gitlab, a new `pcf` file is available, with pin names/assignments for the SPI interface.

When using `icoprogram`, be sure to unload the `spidev` kernel module first, and reload it afterwards using:

---

```
modprobe spi-bcm2835 -r
...your icoprogram commands...
modprobe spi-bcm2835
```

---

**Note:** If `/dev/spi` cannot be found, be sure to turn on the SPI interface in `raspi-config`! This needs to be done *only* once, to let the `modprobe` workflow as shown before to work as expected.

#### Required hardware: Raspberry Pi

**Assignment 11:** *Show a working SPI interface between the Raspberry Pi (as SPI master, C/C++ is mandatory!) and the FPGA (as SPI slave).*

*Think of a way to demonstrate the connection between the devices (that is, test to make clear that the connection functions correctly). Show the working demonstrators to the Teaching Assistants.*

#### Tips and Remarks

- See the links below for starting points, for both the ARM and FPGA side of the communication.
- If both sides compile/synthesize but it is not working, check if your chip select, clock polarity, phase, etc are the same on both sides.
- Read about the SPI communication protocol if these terms are unfamiliar!
- You could use a python SPI implementation for debugging the FPGA's functionality.
- Look-up the `spi-bcm2835` kernel module (Google it) for more information on its specifics and possible interfacing.

#### Required hardware: Raspberry Pi or Altera DE10-NANO

**Assignment 12:** *During the final exercise, you will construct a vision-in-the-loop demonstrator.*

*Read the final chapters and already think about the final demonstrator. Also think about design choices with respect to the communication between the two processing elements.*

Pointers:

- The communication at the software side can be handled in different ways (polling, interrupts, etc). How does this influence the performance?
- Measure interesting details of your implementation (speed, overhead, resource usage, etc), which can be used later on.

### 6.3 Links

- Verilog SPI example: <https://www.fpga4fun.com/SPI2.html>
- The Raspberry Pi forums talking about spidev and SPI examples: <https://forums.raspberrypi.com/viewtopic.php?t=304828>

## 7 Influence physical system behavior

### 7.1 Goals

- Apply control to your setup
- Figure out the optimal architecture to use for the motor controllers.

### 7.2 Assignments

**Required hardware: Raspberry Pi or Altera DE10-NANO**

**Assignment 13:** *Use a 20-sim controller to control the motor position*

#### Tips and Remarks

- Install 20-sim on your own laptop, 20-sim only works on Windows so Linux and Mac users will have to use a Windows virtual machine.
- A 20-sim model of the setup is available. It is in the RAM gitlab.
- After opening the *simulator* it is possible to generate code (Tools→Real Time Toolbox→C-Code Generation), use '*C-Code for 20-sim submodel*' to generate C code for only *one* submodel (the controller).
- The model itself is available in the `xxmodel.c` and `xxmodel.h` files, read the `xxmodel.c` file to get more information on using the controller.
- It might be a good idea to rename the `xxmodel` files when you export multiple controllers, as they all have the same name and will override each other.
- To actually use the controller code, build an application which provides interface/safety/etc layer(s) between the 20-sim code and the hardware.
- The 20-sim model requires input signals which are defined in radians (SI standard), thus a conversion is required for the encoder values.
- Additionally, for the JIWIY setups a homing procedure is required.
- The model needs to be executed in a time-driven mode, which must match the update frequency of the model in 20-sim.
- The generated 20-sim model (probably) makes use of functions from the math library (available from the `math.h` header), therefore `libm` is required. Make sure that it is used during the linking phase as in: `gcc (...) -lm (...)`

**Required hardware: Raspberry Pi or Altera DE10-NANO**

**Assignment 14:** *After implementing the controller(s) on one of the platforms, how is the performance compared to an implementation on the other platform?*

- Think of relative performance measurements so you may use this for the final assignment.

## 8 Design the final demo

### 8.1 Goals

- Building a working vision-in-the-loop demo with the given setup and the created IP cores.

### 8.2 Assignments

**Required hardware: Raspberry Pi or Altera DE10-NANO**

**Assignment 15:** *Think of a demonstrator and perform the Design-Space Exploration for it*

#### Tips and Remarks

- Think of a demonstration with your setup (for example: spot tracking, but other ideas are encouraged as well!). Do discuss your final idea with a TA.
- Find or create the required algorithms and/or the implementations. Make sure they are not too complex, remember the time frame available.

Things you might want to take into consideration are for example:

- ARM on Raspberry Pi *vs* ARM on DE10-Nano
- Lattice ICE40 *vs* Cyclone 5
- Software *vs* hardware.
- Integer *vs* floating point precision in the control loop.
- Development time *vs* performance.
- Source quality (RAW/JPEG/...) *vs* required processing power.
- *etc.*

**Required hardware: Raspberry Pi or Altera DE10-NANO**

**Assignment 16:** *Implement the most promising result(s) to create your demo*

Use the DSE and the results of the earlier assignments to develop a vision-in-the-loop demonstration for your setup.

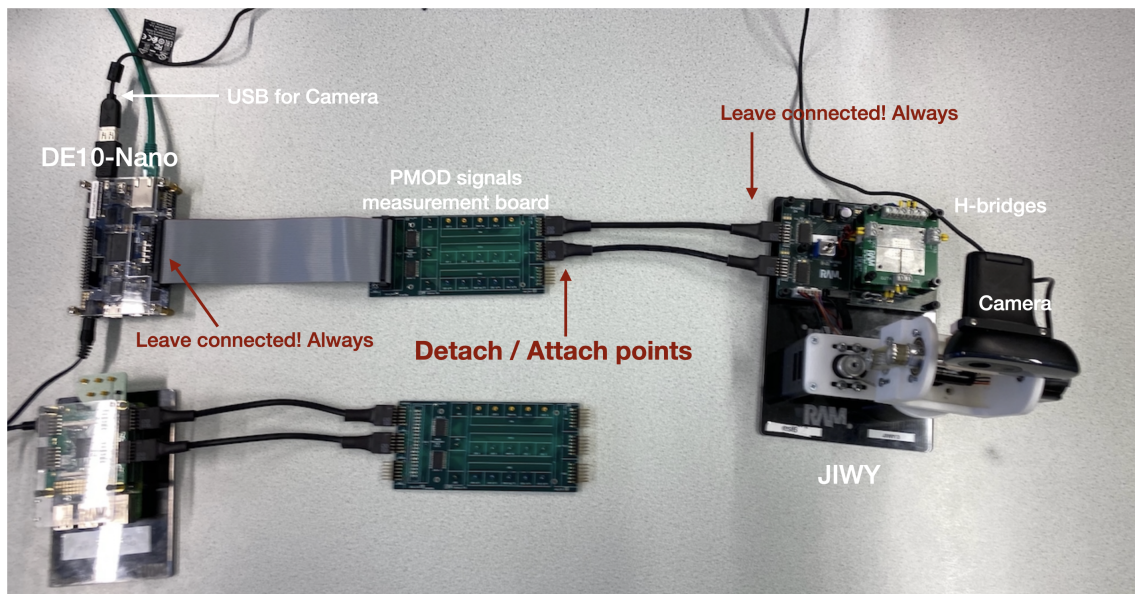
## A Working with the setup

When in the lab, you are provided with the following setup parts:

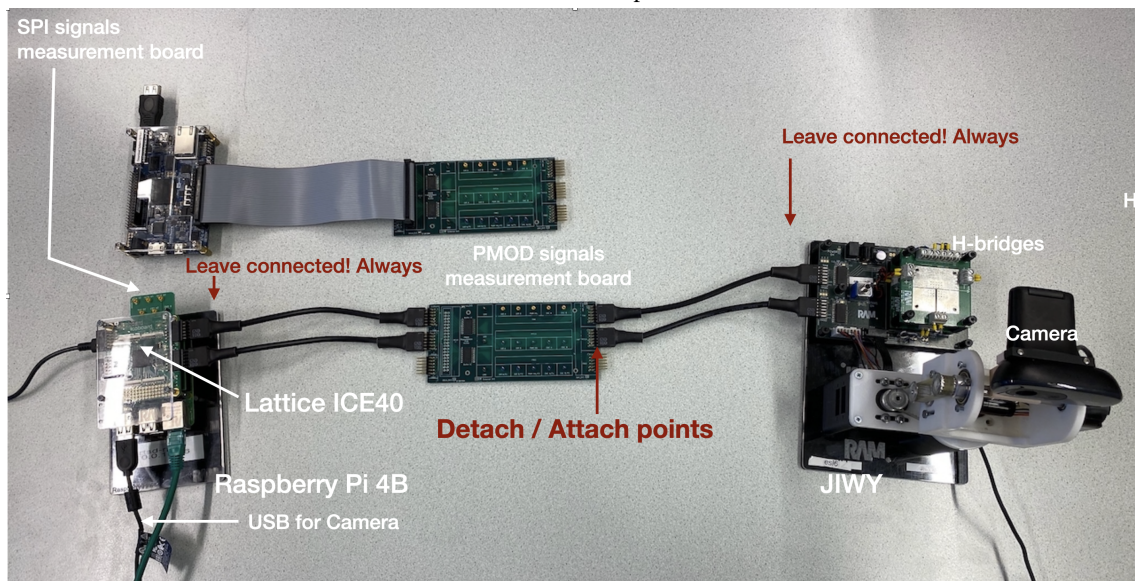
- a DE10 nano connected to a PMOD Measurement PCB + SD Card.
- a Raspberry Pi + IcoBoard with a SPI (GPIO) measurement PCB, connected to a PMOD Measurement PCB + SD Card.
- a JIWIY with PMOD cables connected.

**ONLY disconnect and connect PMOD cables at the side of the Measurement PCBs!** So other cables will *always* stay correctly placed at their connections.

This leads to either of the two setups, as shown in Figure A.1.



(a) DE10 setup



(b) Raspberry Pi & IcoBoard setup

**Figure A.1:** The two setups. **Note:** Only cables between JIWIY and Measurement PCB have to be changed at the side of the Measurement PCB.

## A.1 Connections

### A.1.1 IcoBoard to PMOD Measurement PCB

In the RAM gitlab, the `ico-jiwy.pcf` file is available. This file maps the names of your ports in Verilog to the hardware pins on the IcoBoard. This is also why this pcf is specified with the `--pcf` selector in the `nextpnr` command in Section 2.1.

To get support, using this pin configuration file is mandatory when working with the IcoBoard.

The following pin names are configured, and map to the similarly named pin on the JIWI, as well as the PMOD Measurement PCB. See Table A.1 for the pin names and Figure A.2 for what these pins represent on the JIWI.

| Pitch/Tilt PMOD connector | Yaw/Pan PMOD connector | General |
|---------------------------|------------------------|---------|
| PITCH_DIRA                | YAW_DIRA               | LED     |
| PITCH_DIRB                | YAW_DIRB               |         |
| PITCH_PWM_VAL             | YAW_PWM_VAL            |         |
| PITCH_ENC_A               | YAW_ENC_A              |         |
| PITCH_ENC_B               | YAW_ENC_B              |         |

**Table A.1:** Pin name mapping of the PMOD connectors and LED. The icoBoard's LED used in the blinking LED example.

### A.1.2 DE10 to PMOD Measurement PCB

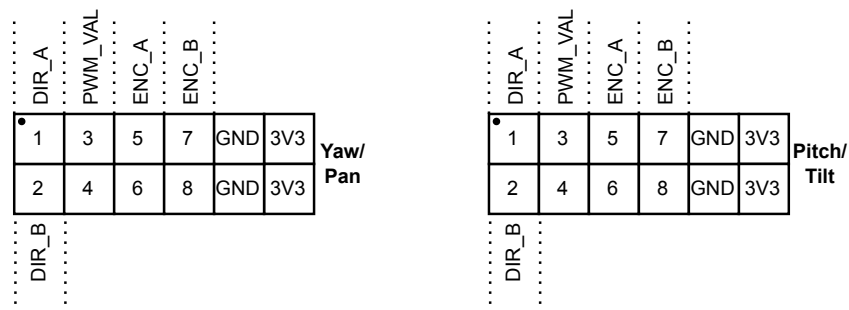
Also a constraint file for the DE10 is available in the RAM gitlab (`DE10_JIWI_PINOUT.csv`). This file can be imported in the Pin Planner in Quartus.

It uses the same naming as the IcoBoard above in Table A.1, except, as there are 8 LEDs, they are named `LED[0]` upto `LED[7]`.

**Note 1** Changing from Raspberry Pi/IcoBoard to DE10 (and vice versa) means that you change cables / PMOD connectors *only* at the PMOD measurement PCB side of the **JIWI** cables.

**Note 2** The motors can be switched off to be sure that no sudden, unintended movements occur using a mechanical switch on the JIWI board.

The signal connectors on the JIWI are called PMOD connectors; they have a more or less standardized pinout ([PMOD specification](#)). The pinout of the JIWI is shown in Figure A.2.



**Figure A.2:** Pinout of JIWI signal connectors.

## B Verilog Simulation

For Verilog simulation, `iverilog` and `gtkwave` can be used. In this Appendix, a general workflow is presented.

### B.1 Installing the tools

On a Linux system, use:

---

```
sudo apt update
sudo apt install gtkwave iverilog
```

---

### B.2 Using the tools

To use `iverilog` and `gtkwave`, be sure to include the following two lines right after your `initial begin` of your test bench.

---

#### initial begin

```
$dumpfile("signals.vcd"); // Name of the signal dump file
$dumpvars(0, adder_tb); // Signals to dump. If a module is specified, all its signals are dumped
```

---

For this example, we will use two files, `adder.v` and `adder_tb.v`.

---

```
module adder(input [7:0] a, b, output [7:0] sum);
    assign sum = a + b;
endmodule
```

---

**Listing B.1:** `adder.v`

---

```
`include "adder.v"

module adder_tb;
    reg [7:0] a, b;
    wire [7:0] sum;

    // instantiate DUT
    adder dut(
        .a(a),
        .b(b),
        .sum(sum)
    );

    // generate input signals
    initial begin
        $dumpfile("signals.vcd"); // Name of the signal dump file
        $dumpvars(0, adder_tb); // Signals to dump
        a = 8'b00000001;
        b = 8'b00000010;
        #10; // delay for 10 time units
        a = 8'b00000100;
        b = 8'b00001000;
        #10; // delay for 10 time units
        a = 8'b00010000;
        b = 8'b00100000;
        #10; // delay for 10 time units
        $finish; // end simulation
    end
```

---

```
end

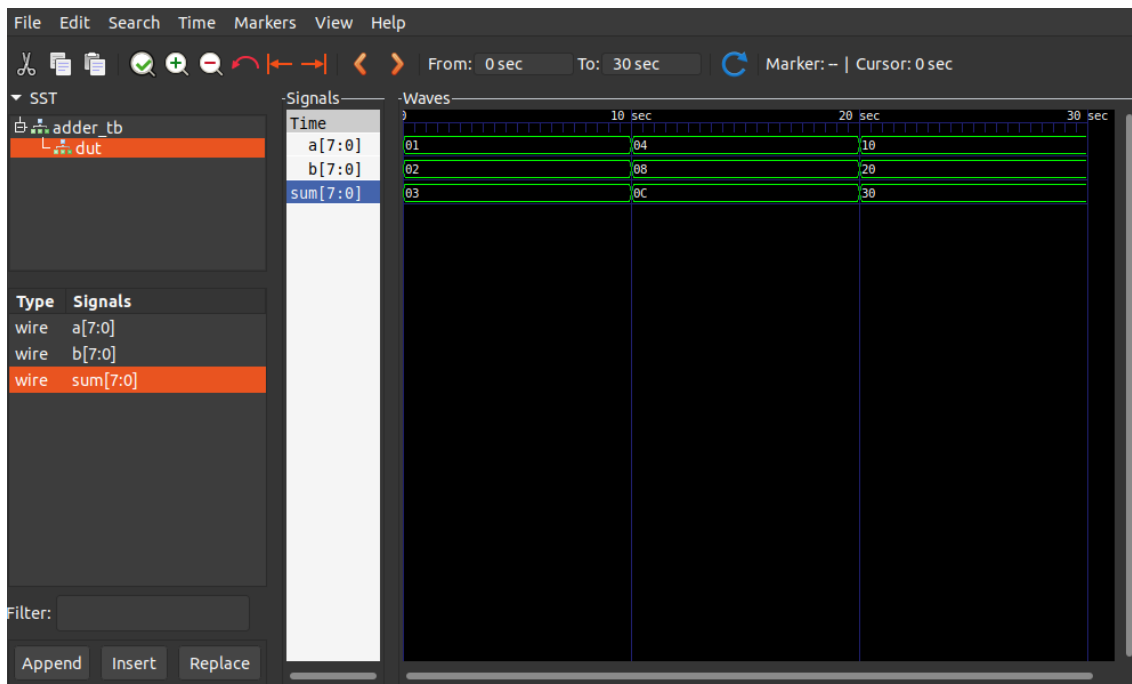
// check output signals
always @ (sum) begin
    $display("sum = %d", sum);
end
endmodule
```

**Listing B.2:** adder\_tb.v

Then, using the following set of commands, gtkwave is opened, from which you can open traces.

```
iverilog -o adder_tb.vvp adder_tb.v
vvp adder_tb.vvp
gtkwave signals.vcd
```

A window should open up, and in the upper-left in the SST field, click on your module, and in the field below click on the wire or register you want to have a trace of. It should then appear in the main trace window. See Figure B.1 for an overview.



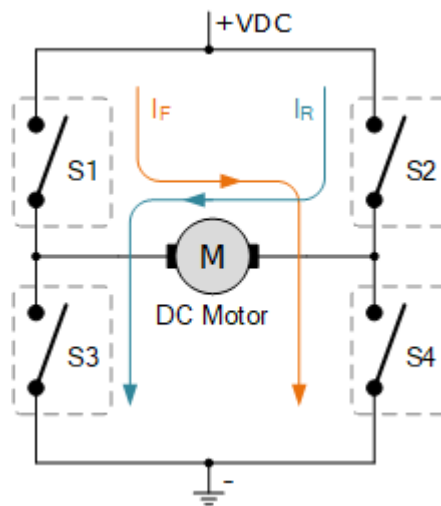
**Figure B.1:** gtkwave window, with the adder\_tb verilog module traces visible

## C H-bridge

An H-Bridge is a circuit that allows a voltage to be applied across a load in either direction. It is commonly used to control the direction of a DC motor. The H-Bridge is a circuit with four switches arranged in an H formation, see Figure C.1. An H-bridge has the following behaviour:

- If S1 and S4 are closed the current flows from left to right through the motor (the orange path)
- If S2 and S3 are closed the current flows from right to left through the motor (the blue path)
- If S1 and S2 or if S3 and S4 are closed the motor brakes
- If all switches are open the motor can spin freely

The motor brakes when S1 and S2, or when S3 and S4 are closed, because a short circuit is created between the terminals of the motor and a strong current generated by the spinning motor flows through the motors internal resistance.



**Figure C.1:** Schematic of an H-bridge. When S1 and S4 are closed current follows the orange path, when S2 and S3 are closed current follows the blue path.

For the JIYW we use two *VNH2SP30-E integrated H-bridge motor drivers* to control the motors. The only thing you need to know for the setup is that it has three inputs ( $IN_A$ ,  $IN_B$  and  $IN_C$ ) and two outputs ( $OUT_A$  and  $OUT_B$ ).

$IN_A$  and  $IN_B$  are the directional signals and control the switches as shown in Figure C.1. If  $IN_A$  is high, S1 is closed and S3 is open and vice versa if  $IN_A$  is low.  $IN_B$  does the same for S2 and S4.  $IN_C$  is a PWM signal to control the speed of the motors.

$OUT_A$  and  $OUT_B$  are connected to both terminals of the motor. If you want to know more about the *VNH2SP30-E integrated H-bridge motor driver* feel free to read its datasheet<sup>1</sup>

<sup>1</sup>Datasheet of VNH2SP30-E integrated H-bridge motor driver: <https://www.pololu.com/file/0J52/vnh2sp30.pdf>

## Bibliography

Student Charter (2020). *Student Charter at University of Twente*. URL: <https://www.utwente.nl/en/ces/sacc/regulations/charter>.